Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет “Высшая школа экономики”

Факультет компьютерных наук
Дисциплина «Архитектура компьютера и операционные системы»

ОТЧЁТ
по индивидуальному заданию № 2

Вариант 31
«Управление взлётно-посадочными полосами»

Выполнил: Шепурев Степан Сергеевич
Группа: БПИ252
Дата: 24.10.2026

Москва, 2026

Содержание

1 Постановка задачи и модельные решения

2 Структура программы и роли потоков

3 Синхронизация и взаимодействие потоков

4 Диспетчеризация и обеспечение корректности

5 Пользовательский интерфейс и ввод данных

6 Сборка запуск и завершение программы

7 Тестирование

8 Результаты демонстрационных запусков

9 Критическая оценка и использование ИИ

Приложение А События и состав проекта

Работа посвящена многопоточной реализации варианта 31. Представлены устройство двух программ, взаимодействие участников, способы запуска и остановки, фактические результаты проверок и ограничения решения.

1 Постановка задачи и модельные решения

Цель индивидуального задания № 2 — разработать консольную многопоточную имитационную программу для Linux на языке C с использованием libc, системных вызовов и pthread. За основу взята предметная модель первой работы; поведение самолётов, полос и диспетчера теперь выполняется в самостоятельных потоках.

Условие варианта 31

Аэропорт располагает одной или несколькими взлётно-посадочными полосами. Самолёты поступают через случайные интервалы и запрашивают посадку или взлёт. Каждый самолёт имеет уникальный номер, тип, требуемую длину полосы, продолжительность операции и категорию приоритета.

Прибывающий самолёт, не получивший полосу, остаётся в зоне ожидания и расходует топливо. Вылетающий самолёт ждёт на земле. Диспетчер назначает совместимую свободную полосу с учётом интервала безопасности. Критический остаток топлива даёт аварийный приоритет, однако уже начатая операция не прерывается.

Входные данные определяют полосы, типы и количество самолётов, интервалы заявок, длительности операций, топливо и расход, приоритеты, интервалы безопасности и закрытия. Требуется отображать все ключевые события и завершать моделирование после обработки заявок, окончания смены либо прерывания пользователем. Источник условия — Task.pdf, вариант 31, страницы 60–61.

Принятые допущения

Время дискретное: один шаг соответствует одному модельному такту. Длительности и интервалы задаются в тактах, длины полос — в метрах. Топливо измеряется в условных единицах, расход — в единицах за такт ожидания. Значение delay-ms регулирует скорость показа событий и не является длительностью реального взлёта.

Типы самолётов и закрытия представлены данными. FLOW заранее формирует конечный набор заявок; MAX_TICKS=0 снимает ограничение смены, но не создаёт бесконечный поток новых самолётов. Один самолёт выполняет только одну запрошенную операцию.

Топливо трактуется как допустимый бюджет ожидания. При его исчерпании регистрируется уход на запасной аэродром; сам перелёт и физический резерв топлива не моделируются. Порядок выполнения потоков определяет планировщик ОС, поэтому точное расписание повторного запуска может отличаться.

2 Структура программы и роли потоков

Файл	Назначение
src/airport.h	Структуры Plane, Runway, Dispatcher, Notification, константы и объявления функций.
src/main.c	Разбор CLI, настройка журналирования и сигналов, запуск модели, код завершения.
src/config.c	Чтение конфигурации, проверки входа, создание самолётов из AIRCRAFT и FLOW.
src/simulation.c	Потоки участников, время, диспетчеризация, топливо, закрытия, инварианты и статистика.
src/sync.c	Создание, ожидание, отправка и уничтожение уведомлений для двух вариантов синхронизации.
src/io.c	Согласованный вывод событий, запись через файловые дескрипторы и обработка ошибок.
CMakeLists.txt	Сборка airport_sem и airport_cond из пяти файлов C; подключение Threads::Threads.

Соответствие участников потокам

Участник	Поведение
Каждый самолёт	Подаёт заявку, переходит в очередь, обновляет топливо в ожидании и уведомляет диспетчера.
Каждая ВПП	Отслеживает закрытия, проверяет назначение перед стартом, завершает операцию и освобождает полосу.
Диспетчер	Выбирает самолёты из зарегистрированной очереди и резервирует подходящие полосы.
Главный поток	Создаёт участников, продвигает часы, принимает сигналы, проверяет состояние и организует остановку.

Для N самолётов и R полос создаётся N + R + 1 рабочих потоков. Вместе с главным потоков в процессе N + R + 2. В основном сценарии N=30 и R=3: создаются 34 рабочих потока, всего — 35. Завершившие работу самолёты могут выйти раньше остальных участников.

Объекты находятся в общих массивах. Индексы связывают самолёт с полосой и полосу с самолётом. Максимальные размеры: 300 самолётов, 10 полос, 20 типов, 30 закрытий и 20 описаний FLOW. Ограничения проверяются при чтении данных.

3 Синхронизация и взаимодействие потоков

Основная программа с семафорами

airport_sem использует общий мьютекс world и отдельное уведомление Notification для каждого рабочего потока. world защищает состояние самолётов и полос, часы, статистику и вывод событий. Назначение самолёта и изменение состояния полосы выполняются в одной критической секции: другой поток не видит частично выполненное назначение.

Уведомление включает мьютекс, флаг pending и семафор. Отправитель под локальным мьютексом устанавливает pending и вызывает sem_post, если уведомление ещё не ожидает обработки. Получатель блокируется в sem_wait; после пробуждения сбрасывает pending под локальным мьютексом. Ожидание повторяется при EINTR.

Уведомление означает «проверь актуальное состояние», а сами заявки хранятся в общей модели. Несколько уведомлений объединяются в одно. Это не теряет самолёты и не требует отдельного семафорного счётчика на каждое изменение топлива.

Дополнительная программа с условными переменными

airport_cond собирается с определением USE_CONDVAR. В Notification вместо семафора используется pthread_cond_t. Отправитель меняет pending под мьютексом и вызывает pthread_cond_signal. Получатель ожидает в цикле while (!pending) через pthread_cond_wait, что учитывает возможные ложные пробуждения.

Флаг pending сохраняет уведомление, пришедшее до начала ожидания. Операции с состояниями аэропорта и предметные правила одинаковы у двух программ; различие сосредоточено в sync.c и определении Notification.

Сценарий взаимодействия

Главный поток продвигает модельное время и уведомляет участников. Поток самолёта регистрирует REQUEST и QUEUE, затем уведомляет диспетчера. Диспетчер обновляет состояние ожидающих, выбирает самолёт и резервирует ВПП, после чего будит её поток. Поток ВПП проверяет условия START, выполняет переход в RUNNING, а по достижении времени окончания фиксирует FINISH и RELEASE и снова уведомляет диспетчера.

Общего барьера между тактами нет: главный поток не ждёт окончания всех действий текущего такта. Потоки могут проснуться позднее плановой метки. Поэтому FINISH возможен позже заданной границы, но не раньше START + duration.

Ожидание уведомления выполняется без world. При вложенных блокировках порядок один: world, затем мьютекс уведомления. После ожидания локальный мьютекс освобождается до захвата world. Это исключает цикл ожидания этих двух видов блокировок в штатной работе.

4 Диспетчеризация и обеспечение корректности

Состояния самолёта и правила назначения

```
PENDING -> WAITING -> ASSIGNED -> RUNNING -> DONE
             |           |
             v           +-> WAITING при отмене резерва
          DIVERTED
```

ASSIGNED означает резерв полосы, а не начавшуюся операцию. Перед START полоса повторно учитывает топливо за ожидание и проверяет закрытие. При закрытии резерв снимается, самолёт возвращается в очередь с REQUEUE. При исчерпании топлива фиксируется DIVERT. Из RUNNING начатая операция не вытесняется.

Очередность определяется аварийным состоянием, затем категорией HIGH > NORMAL > LOW, затем более ранним request_tick и индексом записи. Выбор действует среди уже зарегистрированных ожидающих самолётов, для которых сейчас есть доступная ВПП. Выбирается самая короткая подходящая свободная открытая полоса, прошедшая интервал безопасности.

После фактического FINISH в такт T назначается next_free_tick = T + safety. Закрытие действует на полуинтервале [start, end); пересекающиеся закрытия учитываются совместно. Если ВПП закрылась во время RUNNING, операция продолжается, но новые старты запрещены.

Топливо прибывающего уменьшается на burn × число прошедших тактов ожидания. Повторный расчёт в том же такте не списывает его второй раз. При fuel ≤ burn × EMERGENCY_FUEL_TICKS включается аварийный приоритет, при нуле — DIVERT. Диспетчер также обновляет топливо перед выбором, чтобы не использовать устаревшие данные.

Инварианты

Требование	**Способ обеспечения**
Не более одного самолёта на ВПП	Один plane_index; резервирование под world; проверка взаимных ссылок.
Совместимость по длине	Проверка при выборе и в check_invariants.
Интервал безопасности	Назначение запрещено до next_free_tick; тест сравнивает START и RELEASE.
Операция не повторяется	Переходы состояний и проверка times_started ≤ 1.
Явный исход по топливу	События EMERGENCY и DIVERT; топливо ограничивается снизу нулём.

check_invariants вызывается главным потоком под world. Нарушение приводит к INVARIANT_FAILURE и коду 3. Уникальность номера проверяется при вводе; проверка интервала безопасности обеспечена алгоритмом и тестом, а не отдельным утверждением внутри check_invariants.

5 Пользовательский интерфейс и ввод данных

Интерфейс состоит из параметров командной строки, текстового файла сценария и журнала событий. По умолчанию используется configs/default.conf. После чтения файла применяются переопределения CLI. Комментарии в конфигурации начинаются с #.

Параметр	Назначение
--config, -c	Путь к конфигурационному файлу.
--seed	Начальное значение генератора случайных чисел.
--max-ticks	Последний такт смены; 0 — без ограничения смены.
--delay-ms	Задержка между тактами; при 0 используется пауза 1 мс.
--log	Файл журнала; по умолчанию simulation.log.
--quiet	Отключение вывода событий в терминал.
--no-log	Отключение файла журнала.
--help	Вывод справки.

Пример конфигурации

```
SEED 31
MAX_TICKS 30
DELAY_MS 100
EMERGENCY_FUEL_TICKS 3
RUNWAY R1 2500 2
TYPE A320 1800 3 3
CLOSE R1 10 15
AIRCRAFT A001 ARRIVAL 0 A320 12 1 NORMAL
AIRCRAFT D001 DEPARTURE 1 A320 0 0 HIGH
```

RUNWAY задаёт номер, длину и safety; TYPE — имя, требуемую длину, время посадки и взлёта. AIRCRAFT задаёт номер, операцию, такт заявки, тип, топливо, расход и приоритет. CLOSE содержит имя ВПП, начало и конец закрытия. Сначала описываются объекты, затем записи, которые на них ссылаются.

FLOW задаёт операцию, количество самолётов, такт первой заявки и минимальный/максимальный интервал между следующими. Для генерации используются PRIORITY_WEIGHTS, ARRIVAL_FUEL и ARRIVAL_BURN. Seed воспроизводит входной набор в одной реализации libc; он не фиксирует расписание потоков.

Ввод и вывод

Конфигурация открывается через open, затем читается fdopen/fgets. Числа проверяются через strtol: отвергаются лишние символы, переполнение и недопустимые значения. Также отвергаются неизвестные команды и ссылки, лишние поля, дубликаты номеров и превышение размеров массивов.

Журнал открывается через open, события выводятся через write, файл закрывается close. Запись учитывает частичные результаты и EINTR. Строка события содержит модельный такт, тег и описание участника; в конце печатаются SUMMARY и сведения о незавершённых самолётах.

6 Сборка запуск и завершение программы

Требуются Linux, компилятор с поддержкой C11, CMake 3.16 или новее и pthread. На Windows проект собирается в Ubuntu через WSL. MinGW не подходит для текущей Linux-реализации с sigtimedwait и POSIX-сигналами.

В PowerShell нужно запустить wsl -d Ubuntu, затем перейти в папку проекта. Дальнейшие команды выполняются в Linux из корня проекта:

```
cmake -S . -B build-wsl
cmake --build build-wsl -j
./build-wsl/airport_sem -c configs/default.conf --delay-ms 100
./build-wsl/airport_cond -c configs/default.conf --delay-ms 100
bash scripts/run_tests.sh build-wsl
```

В CLion следует выбрать инструментальную цепочку WSL и отдельный каталог сборки для Linux. Кэш старой сборки MinGW нельзя использовать как Linux-сборку.

Условия завершения

Условие	Наблюдаемый результат
Все самолёты DONE или DIVERTED	COMPLETE; termination=ALL_REQUESTS_COMPLETED; код 0.
Достигнут MAX_TICKS > 0	SHIFT-END; termination=SHIFT_ENDED; код 0. Выводятся незавершённые самолёты.
SIGINT или SIGTERM	STOP; termination=USER_INTERRUPT; код 130.
Некорректный ввод или открытие журнала	Диагностика; код 2, моделирование не начинается.
Ошибка создания потоков или состояния	Очистка созданных ресурсов; код 3.
Ошибка записи или закрытия журнала	Сигнализация ошибки ввода-вывода; код 4.

Если подходящей полосы нет, прибывающий самолёт расходует топливо и получает DIVERT. Вылетающий продолжает ожидать до окончания смены или сигнала. При MAX_TICKS=0 такое ожидание может быть неограниченным; для остановки используется Ctrl+C.

Безопасная остановка потоков

SIGINT и SIGTERM блокируются главным потоком до pthread_create, поэтому рабочие потоки наследуют блокировку. Сигналы принимает главный поток через sigtimedwait. Он устанавливает stopping, пробуждает всех созданных участников и выполняет pthread_join для каждого. Затем уничтожаются уведомления и общий мьютекс.

При частичном отказе pthread_create завершаются только успешно созданные потоки, уничтожаются только инициализированные уведомления. pthread_cancel не используется. При остановке выполняющаяся операция остаётся незавершённой в статистике. Фатальный отказ уже работающего синхропримитива приводит к abort.

7 Тестирование

Проверка выполнена 24.10.2026 в Ubuntu под WSL2, GCC 15.2.0. Текущие исходные файлы собраны в обе программы. Штатный scripts/run_tests.sh выполнен на рабочей копии проекта; в этом прогоне прошли все 16 запусков — по восемь для каждой реализации синхронизации.

Сценарий	Проверка скрипта	Результат прогона
test_nominal.conf	Итоговая сводка и обычное завершение.	Обе версии прошли.
test_emergency.conf	Наличие события EMERGENCY.	Обе версии прошли; тест нестабилен.
test_closure.conf	Наличие CLOSE и OPEN.	Обе версии прошли.
test_incompatible.conf	DIVERT и окончание смены.	Обе версии прошли.
test_safety.conf	START не раньше метки доступности из RELEASE.	Обе версии прошли.
test_duplicate.conf	Отклонение дубликата с кодом 2.	Обе версии прошли.
test_interrupt.conf + SIGINT	STOP, код 130, 3 созданных и 3 объединённых потока.	Обе версии прошли.
test_interrupt.conf + SIGTERM	Аналогичная корректная остановка.	Обе версии прошли.

Границы проверок

Успешный единичный прогон не гарантирует успех всех повторов. При предыдущей повторной проверке test_emergency.conf обнаружено расписание, при котором A001 получает полосу раньше D001 и успевает сесть до критического расхода топлива. Модель завершается корректно, но проверка наличия EMERGENCY не проходит. Такое поведение наблюдалось у обеих программ.

Причина — одновременные заявки в такт 0 и отсутствие гарантии их одновременной регистрации. Для устойчивого теста нужно сначала накопить ожидающих самолётов при недоступной полосе, затем проверять аварийное состояние и фактический порядок назначения. Это направление доработки теста, а не выполненное исправление.

Проверка CLOSE/OPEN подтверждает события, но сама по себе не доказывает отсутствие START внутри закрытия. Для полного контроля нужна дополнительная проверка интервалов по журналу. Текущий скрипт также не перебирает все возможные расписания ОС.

Результаты CTest и санитайзеров из отчёта первой работы относятся к прежней последовательной программе. В качестве подтверждения текущей версии здесь используются фактически выполненные Linux-сборка, штатные сценарии и два демонстрационных запуска.

8 Результаты демонстрационных запусков

Для обеих программ выполнен запуск configs/default.conf с seed=31 и delay-ms=10. Сценарий содержит 30 самолётов: 16 прибывающих и 14 вылетающих; 3 ВПП, 5 типов и 2 окна закрытия. Длина RWY-A — 3400 м, RWY-B — 2500 м, RWY-C — 1900 м; safety соответственно 2, 1 и 1 такт.

Показатель	**airport_sem**	**airport_cond**
Причина завершения	Все заявки обработаны	Все заявки обработаны
Последний такт	63	63
Всего самолётов	30	30
Посадки	10	11
Взлёты	14	14
Уходы на запасной	6	5
Ожидающие / выполняющие / будущие	0 / 0 / 0	0 / 0 / 0
Назначения / аварийные назначения	24 / 1	25 / 2
Операции RWY-A / RWY-B / RWY-C	9 / 10 / 5	9 / 11 / 5
Создано / объединено рабочих потоков	34 / 34	34 / 34

Для airport_sem баланс исходов: 10 + 14 + 6 = 30; для airport_cond: 11 + 14 + 5 = 30. Число операций полос равно числу завершённых посадок и взлётов. Уходы на запасной — допустимый исход при конечном топливе, очередях и ограниченной пропускной способности.

Фрагмент журнала airport_sem

```
[T=0000] [QUEUE    ] GA001 entered waiting queue
[T=0000] [ASSIGN   ] ATC assigned GA001 to RWY-C
[T=0000] [ASSIGN   ] ATC assigned GD001 to RWY-A
[T=0000] [START    ] GD001 started TAKEOFF on RWY-A; finish at T=5
[T=0000] [START    ] GA001 started LANDING on RWY-C; finish at T=3
```

Фрагмент показывает работу разных полос в одном модельном интервале. Операция на каждой ВПП исполняется её потоком; изменения общих данных выполняются поочерёдно под мьютексом.

Различие числа посадок объясняется порядком регистрации заявок и пробуждения участников. Seed фиксирует генерацию входов, но не работу планировщика ОС. Приведённые числа описывают эти два запуска; они не являются обязательным эталоном для каждого повтора и не доказывают преимущество одной реализации по быстродействию.

9 Критическая оценка и использование ИИ

Использование ИИ и внесённые изменения

При подготовке проекта использовался: для анализа задания, тестов, проверки результатов и подготовки документации. Запросы в report/AI_PROMPTS.md.

По сравнению с первой работой последовательные вызовы ролей заменены самостоятельными pthread. Добавлены world, уведомления участников и отдельное состояние ASSIGNED, отделяющее резерв от начала операции. Подготовлены два исполняемых файла с общей логикой и различными примитивами ожидания.

Для корректной многопоточности топливо учитывается по разности модельного времени; диспетчер обновляет его перед выбором. Перед START повторно проверяются закрытие и остаток топлива. Сигналы обрабатываются главным потоком через sigtimedwait, а выход завершается пробуждением и pthread_join участников.

Ограничения и необходимые улучшения

Общий мьютекс упрощает согласование модели, но сериализует изменения и запись событий. Медленный вывод способен задерживать участников. На каждом такте уведомляются все потоки, включая те, которым пока нечего делать. Решение ориентировано на ограниченную учебную модель, а не на максимальную производительность.

В текущей копии проекта многие поясняющие комментарии к синхронизации удалены. Для дополнительного критерия по комментариям полезно восстановить объяснения world, pending, учёта топлива и порядка завершения. Обнаруженная нестабильность аварийного теста требует доработки сценария; простой поиск тега не проверяет все правила приоритета.

Приложение А События и состав проекта

Событие	Тег журнала
Запрос посадки или взлёта	REQUEST
Постановка в очередь	QUEUE
Изменение топлива	FUEL
Назначение полосы	ASSIGN
Начало операции	START
Окончание операции	FINISH
Освобождение полосы	RELEASE
Аварийный приоритет	EMERGENCY
Уход на запасной аэродром	DIVERT
Закрытие и открытие полосы	CLOSE / OPEN
Возврат после отмены резерва	REQUEUE
Штатное завершение / смена / сигнал	COMPLETE / SHIFT-END / STOP

Состав проекта

```
airport_runway_hw2/
  CMakeLists.txt
  README.md
  src/
    airport.h
    main.c
    config.c
    io.c
    simulation.c
    sync.c
  configs/
    default.conf
    test_*.conf
  scripts/
    run_tests.sh
  logs/
    demo.log
  report/
    AI_PROMPTS.md
    test_results.txt
    материалы отчёта
```

После выполнения тестового скрипта в logs/tests появляются журналы сценариев. Каталог build-wsl содержит результаты локальной сборки; он не является исходным кодом. Для защиты достаточно объяснить роли потоков, общий мьютекс, уведомления, выбор ВПП, учёт топлива и последовательность остановки.